

Step 1 — A Single LCEL Chain That Answers One Question, No Tools — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain), **Advanced** (how a chain that other steps will keep building on should actually be written). Read Basic first even if you already know LangChain — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

"LCEL" just means "connect pieces together with `|`," the same way you'd pipe commands in a terminal. Three pieces, chained: a prompt template, the model, and something that turns the model's raw response into plain text.

This step has no tools and no loop — it's just proving that "text goes in, model answers, text comes out" works, before anything more complicated gets layered on top in Step 2. Don't reach for an agent here at all — a plain chain is the right tool for a question the model can already answer from what it knows.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The function to build:

```
def build_simple_chain() -> Runnable:
```

It should assemble three LangChain pieces with `|`: - A `ChatPromptTemplate` — describes the shape of the input (a system message plus a `{question}` placeholder). - A `ChatOpenAI` instance — the actual model call. - A `StrOutputParser` — pulls the plain string answer out of the model's response object, so callers get back `str`, not a LangChain message object.

The result of `prompt | model | parser` is itself a single `Runnable` you can call with `.invoke({"question": "..."})`. Sketch the three pieces and how `|` connects them before writing the function body.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The exercise only asks for a chain that works once, run by hand. But this exact function is what Step 2's agent loop imports and wraps — so it's worth asking now: what would make this chain something you'd actually keep in a real project, not just a demo?

Two things, specifically. First, the model name and temperature shouldn't be buried as string literals inside `build_simple_chain()` — every other step in this project (and Project 3, which reuses this chain again) would have to hunt down and edit this one function to change either. Read them from Doc01's `Config` instead, with a sane default if the config doesn't set them. Second, a live API call can fail transiently — a dropped connection, a momentary rate limit — and a chain with no retry behavior turns a blip into a hard crash. LangChain gives every `Runnable` a `.with_retry()` method for exactly this, for free.

Neither of these changes what the chain *does* — same prompt, same three pieces, same `|`. They change whether it survives being reused for the rest of this project without someone having to come back and harden it later.

The extra pieces: - `config: Config` as a parameter to `build_simple_chain()`, read via `config.chat_model / config.temperature` (add these fields to Doc01's `Config` if they're not already there, with defaults like `"gpt-4o-mini"` and `0.0`). -

`ChatOpenAI(model=config.chat_model, temperature=config.temperature) .-`
`.with_retry(stop_after_attempt=3)` appended after the chain is assembled — it wraps the whole `Runnable`, not just the model call.

Sketch how `config` would flow from `main.py` into `build_simple_chain()` before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic describes the plain idea for a chain that only ever needs to run once, by hand, to prove it works. Intermediate shows the exact, correctly-typed LangChain syntax for that same one-off case. Advanced treats it as what it actually is — a piece of shared infrastructure every later step reuses — and asks what a function reused that many times needs: configurability instead of hardcoded literals, and resilience to the kind of failure a live API call can have that a single manual test run would never happen to hit.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a prompt template with a system message and a place for the question

build the model connection

build a parser that turns the model's reply into plain text

connect all three with | into one chain

in main.py:
    call the chain with a research question
    print the answer
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_simple_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a careful research assistant."),
        ("human", "{question}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini")
    return prompt | model | StrOutputParser()
```

Expected output if you run just this (nothing calls the chain yet): nothing — defining a function doesn't run it. Add `main.py` — build the chain, call `.invoke({"question": "What is the capital of France?"})`, `print()` the result — to see it print a plain-text answer like `The capital of France is Paris.`

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
chain.py:
def build_simple_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_messages([system message, "{question}"])
    model = ChatOpenAI(model="gpt-4o-mini")
    return prompt | model | StrOutputParser()

main.py:
from config import load_config
from logging_setup import get_logger
from chain import build_simple_chain

chain = build_simple_chain()
answer = chain.invoke({"question": "..."})
print(answer)
```

Notice `build_simple_chain()` takes no arguments (yet) and returns the assembled chain — it's a factory function, not something that runs the chain itself. That separation (build vs. run) is what makes it easy to reuse the same chain from `main.py`, a test file, or Step 2's agent loop later. Write the full typed version yourself, wired to Doc01's `config` and `logger`, before moving on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function build_simple_chain(config) -> Runnable:
    prompt = ChatPromptTemplate.from_messages([system message, "{question}"])
    model = ChatOpenAI(model=config.chat_model, temperature=config.temperature)
    chain = prompt | model | StrOutputParser()
    return chain.with_retry(stop_after_attempt=3)
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable
from config import Config

def build_simple_chain(config: Config) -> Runnable:
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a careful research assistant. Answer only from what y
        ("human", "{question}"),
    ])
    model = ChatOpenAI(model=config.chat_model, temperature=config.temperature)
    chain = prompt | model | StrOutputParser()
    # your turn: wrap `chain` with .with_retry(...) before returning it
    ...
```

Fill in the retry wrapping yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same three pieces (prompt, model, parser) connected the same way at 3 completeness levels — Basic just gets it running once with everything hardcoded, Intermediate adds the type contract and Doc01's config/logger habit while the model name is still fixed, and Advanced makes the model name, temperature, and failure behavior all configurable from outside the function, which is what actually matters once several later steps depend on this one function working correctly every time they call it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)